

首先再次回顾虚拟内存中的关键数据结构：页表。如 4.1.4 节所述，页表项中除了记录物理页号外，还记录了属性位，包括用于标识虚拟页访问权限的位（比如是否可写、是否可执行）等。写时拷贝正是利用表示“是否可写”的权限位来实现的。

如图 4.12 下半部分所示，写时拷贝技术允许应用程序 A 和应用程序 B 以只读的方式（在页表项中设置只读权限）共享同一段物理内存。一旦某个应用程序对该内存区域进行修改，就会触发访问权限异常。在异常触发后，CPU 同样会将控制流传递给操作系统预先设置的异常处理函数。在该函数中，操作系统会发现当前异常是由于应用程序写了只读内存，而且相应的内存区域又被标记成写时拷贝（例如，Linux 在上文提及的 `vm_area_struct` 结构体中进行标记）。于是，操作系统会将缺页异常对应的物理页重新拷贝一份，并且将新拷贝的物理页以可读可写的方式重新映射给触发异常的应用程序，此后再恢复应用程序的执行。

针对刚刚提到的两个场景，写时拷贝技术一方面能够节约物理内存资源，比如不同的应用程序以写时拷贝的方式映射相同动态链接库的可写部分（动态链接库的只读部分以只读方式在不同应用程序间共享即可）；另一方面，利用写时拷贝可以让父子程序以只读的方式共享全部内存数据，避免内存拷贝操作带来的时间和空间开销。

大页

前文提到，提高 TLB 命中率可以降低地址翻译的性能开销。由于翻译每个内存页都需要占用一个 TLB 缓存项，因此 CPU 中有限的 TLB 缓存项显得弥足珍贵。在内存页大小为 4KB 的情况下，访问 2MB 内存就需要 512 个 TLB 缓存项。随着应用程序对内存容量需求的变大，CPU 中 TLB 缓存项的数量较难保证高 TLB 命中率。

4.1.4 节介绍的大页（Huge Page）机制能够有效缓解 TLB 缓存项不够用的问题。大页的大小可以是 2MB、1GB 甚至 512GB。相比于 4KB 大小的页面，使用大页可以大幅度减少 TLB 的占用量（如访问 2MB 内存只需要 1 个 TLB 缓存项）。

因此，操作系统可利用硬件提供的大页支持，在添加页表映射时以大页进行映射。此外，Linux 还提供了透明大页（Transparent Huge Page）机制，能够自动地将一个应用程序中连续的 4KB 内存页合并成 2MB 的内存页。使用大页的好处主要包括两方面：一方面，它能够减少 TLB 缓存项的使用，从而有机会提高 TLB 命中率；另一方面，它可以减少页表的级数，从而提升查询页表的效率。不过，大页的使用也会增加操作系统管理内存的复杂度，且过度的大页使用还可能存在资源浪费的问题，例如应用程序可能未使用整个大页而造成物理内存资源浪费。

小知识：最小页面大小

除了大页机制外，AArch64 体系结构还支持多种最小页大小，包括 4KB、16KB、64KB。操作系统可以通过 `TCR_EL1` 寄存器进行配置，选择需要的大小。

练习

- 4.14 前文提及，当操作系统选择 4KB 作为最小页大小时，L2 页表项和 L1 页表项对应大页的页面大小分别是 2MB 和 1GB。那么当操作系统选择 16KB 或 64KB 作为最小页大小时，对应大页的页面大小是多少呢？（注意，在 ARMv8.0 上，当使用 16KB 或 64KB 页面大小的时候，只有 L2 页表项支持大页功能。）
- 4.15 操作系统的页表是否适合使用大页进行映射？

4.3 案例分析：ChCore 虚拟内存管理

4.3.1 ChCore 内核页表初始化

由于需要为应用程序提供虚拟内存抽象，操作系统启动期间会启用 CPU 的虚拟内存功能（启用 MMU）。在启用后，MMU 也会对操作系统执行期间使用的地址进行翻译，因此操作系统在启用 MMU 之前需要首先初始化自己的页表。

通常，操作系统对页表的配置分为两个阶段：第一阶段是启动初期，此时操作系统运行在低地址区域，页表将虚拟地址映射为完全相同的物理地址（ $VA=PA$ ）；第二阶段是在跳转到高地址运行后，页表将虚拟地址映射为物理地址加上固定偏移（ $VA=PA+ \text{偏移量}$ ）。下面以本书配套的实验内核 ChCore 在树莓派（AArch64 体系结构）上的页表初始化流程为例，讲解为什么操作系统的页表通常如上进行配置。

如代码片段 4.11 所示，ChCore 在启动后会先把 CPU 特权级别设置为 EL1（对应内核态），然后设置栈帧并调用 C 代码编写的初始化函数。在该函数中，ChCore 首先配置自己的页表（`init_kernel_pt` 函数），其中会配置两份页表（`boot_ttbr0_10` 负责低地址范围的映射，而 `boot_ttbr1_10` 对应高地址范围的映射），然后启用 MMU。代码片段 4.12 展示了如何启用 MMU，`el1_mmu_activate` 函数首先在两个页表基地址寄存器中写入页表基地址，然后将系统寄存器 `sctlr_el1` 的第 0 位设置为 1，即 `SCTLR_EL1_M` 位，从而开启 MMU。注意，从启动第一条指令到 `el1_mmu_activate` 函数中的 `isb` 指令之间，MMU 都未开启，ChCore 使用的是物理地址；而从 `isb` 指令开始，由于 MMU 已经开启，指令地址也会经过地址翻译。假设 `isb` 指令的上一条 `msr` 指令所在的物理地址是 `0x81000`（即存放在物理内存中的位置），那么 `isb` 指令的物理地址则是 `0x81004`。CPU 在执行 `msr` 指令时，PC 中存放的指令地址是 `0x81000`，由于 MMU 尚未启用，CPU 是通过物理地址 `0x81000` 取出该指令的。CPU 执行完 `msr` 指令后，PC 中存放的指令地址是 `0x81004`（准备执行 `isb` 指令），由于 MMU 已经启用，CPU 中的 MMU 会对 PC 中的地址进行地址翻译（把 `0x81004` 作为虚拟地址），而 `isb` 指令的物理地址是 `0x81004`，所以页表中的地址映射需要虚拟地址与物理地址完全相同。

这就是操作系统页表配置中低地址范围内虚拟地址和物理地址需要完全相同的原因。

代码片段 4.11 ChCore 内核启动流程示例

```

1 BEGIN_FUNC(_start)
2 // 内核运行的第一条指令：获取 CPU ID
3 mrs x8, mpidr_el1
4 and x8, x8, #0xFF
5 // 若当前是 0 号 CPU，则跳转到主 CPU 初始化处执行
6 cbz x8, primary
7 ...
8
9 primary:
10
11 // 设置特权级别为 EL1，即进入内核态执行
12 bl change_el_to_el1
13
14 // 设置内核启动期间的栈帧
15 adr x0, boot_cpu_stack
16 add x0, x0, #INIT_STACK_SIZE
17 mov sp, x0
18
19 // 调用 C 代码编写的初始化函数
20 bl init
21
22 // init 函数不返回，控制流不会到这里
23 b .
24 END_FUNC(_start)
25
26
27 void init(void)
28 {
29 ...
30
31 // 初始化内核页表
32 init_kernel_pt();
33
34 // 启用 MMU
35 ell_mmu_activate();
36
37 // 该函数位于高地址，且不返回
38 do_other_initialization();
39
40 // 控制流不会到这里
41 }

```

代码片段 4.12 ChCore 内核启动时启用 MMU 的代码

```

1 #define SCTL_EL1_M (1)
2
3 BEGIN_FUNC(ell_mmu_activate)

```



```

4    ...
5
6    // 设置页表基地址寄存器
7    adrp    x8, boot_ttbr0_10
8    msr     ttbr0_el1, x8
9    adrp    x8, boot_ttbr1_10
10   msr     ttbr1_el1, x8
11   ...
12
13   mrs     x8, sctlr_el1
14   ...
15   orr     x8, x8, #SCTLR_EL1_M
16   msr     sctlr_el1, x8
17   isb
18
19   ...
20   END_FUNC(ell_mmu_activate)

```

操作系统启动时的第一条指令一般在较低的地址，这是来自硬件 / 固件的要求。通过学习 4.2.1 节可知，应用程序通常使用低地址，而操作系统使用高地址，所以操作系统启动期间需要跳转到高地址运行。如代码片段 4.11 所示，ChCore 在启用 MMU 后，会跳转到高地址继续执行剩余的初始化操作，此后将使用 TTBR1_EL1 中存储的页表进行翻译。对于高地址范围（TTBR1_EL1 对应的翻译），操作系统通常选择固定偏移（虚拟地址等于物理地址加上固定偏移）的映射方式，原因有两个：第一，两条在物理地址上相邻存储但跨越两个物理页的指令，要求对应的虚拟页也相邻，固定偏移映射是能满足该要求的最简单的方式；第二，操作系统通过简单的算术运算即可完成虚拟地址和物理地址之间的转换。

图 4.13 展示了 ChCore 在启动期间使用页表情况的变化过程，主要可以分为三个阶段：第一个阶段不启用 MMU，直接使用物理地址；第二个阶段开启 MMU，运行在低虚拟地址；第三个阶段跳转到高虚拟地址执行。以本书配套实验所用的硬件平台树莓派（3b+）为例，在树莓派上电启动后，固件负责把 ChCore 加载到物理地址 0x80000 处，并且最终跳转到该地址开始执行 ChCore 的第一条指令。此时，ChCore 尚未开启 MMU，因而使用的是物理地址。在经过一些基本初始化流程后，内核调用 `init_kernel_pt` 函数配置操作系统页表 0（负责翻译低地址）和操作系统页表 1（负责翻译高地址）。在操作系统页表 0 中，ChCore 把虚拟地址映射到完全相同的物理地址，即把虚拟地址 0 ~ 1GB 映射到物理地址 0 ~ 1GB（实际代码中映射范围不到 1GB，因为该平台可用的物理内存小于 1GB）。在操作系统页表 1 中，ChCore 把虚拟地址 `KERNEL_VADDR ~ (KERNEL_VADDR+1GB)` 映射到 0 ~ 1GB。之后，ChCore 调用 `ell_mmu_activate` 函数来启用 MMU，该函数首先配置地址翻译相关的控制寄存器，然后把操作系统页表 0 和操作系统页表 1 的基地址分别写入寄存器 TTBR0_EL1 和 TTBR1_EL1，最后把 SCTLTR_EL1 中的

M 位设置为 1，从而使能 MMU（使能页表）。在寄存器 SCTLTR_EL1 设置完成后（下一条指令开始，不妨假设该指令的物理地址是 0x81004），ChCore 使用的地址都不再是物理地址而是虚拟地址（即 CPU 会通过虚拟地址 0x81004 取下一条指令），由于 0x81004 属于低地址范围，所以 MMU 会使用 TTBR0_EL1 进行地址翻译。又由于 TTBR0_EL1 中存储的是操作系统页表 0 的基地址，而该页表把虚拟地址映射到相同的物理地址（即虚拟地址 0x81004 映射到物理地址 0x81004），所以在使能 MMU 后，ChCore 依然能够继续执行。此后，ChCore 跳转到高地址继续其余的初始化工作并且最终启动用户态服务和应用进程。当 ChCore 跳转到高地址执行后，MMU 将使用存储在 TTBR1_EL1 中的操作系统页表 1 进行地址翻译，而不再需要操作系统页表 0。

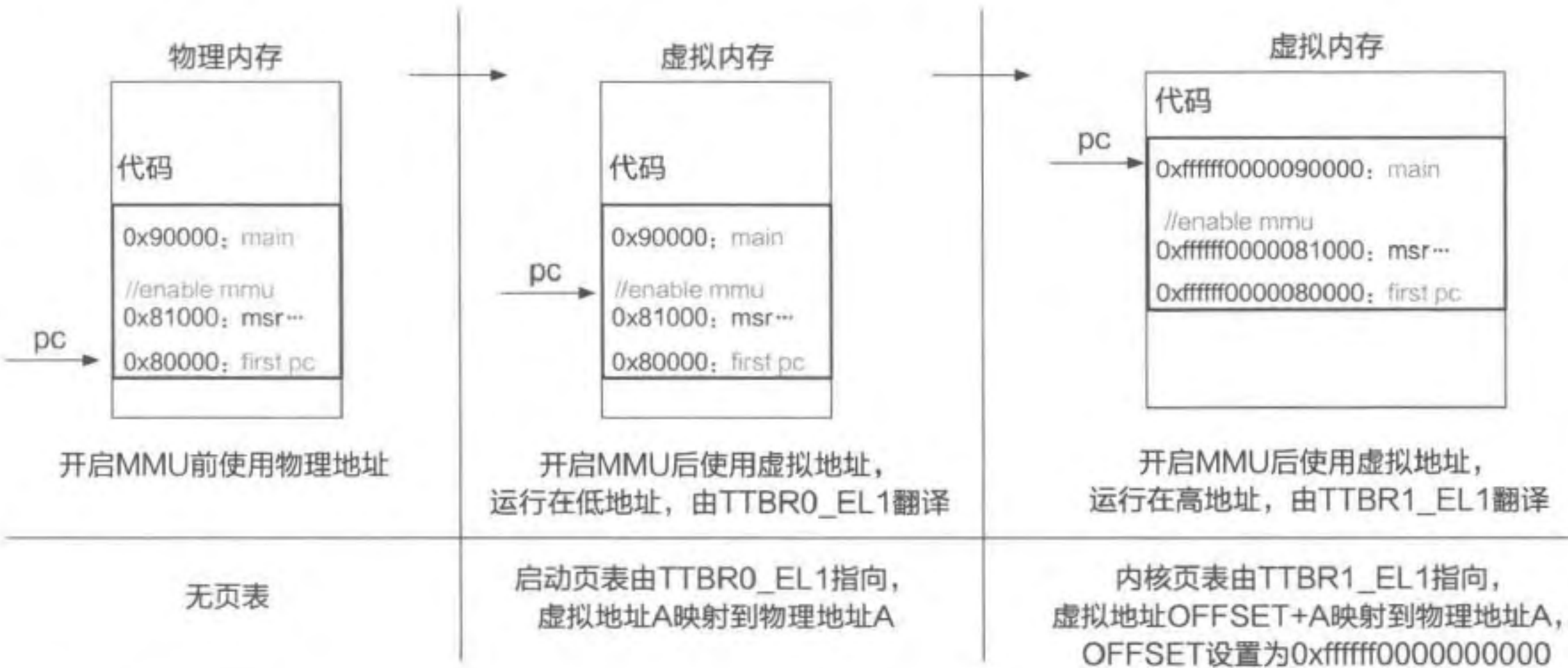


图 4.13 ChCore 启动过程：先使用物理地址，再使用经由启动页表翻译的虚拟地址，之后使用经由内核页表翻译的虚拟地址

4.3.2 ChCore 内存管理

在本书配套的实验内核 ChCore 中，内存管理部分有三个重要的数据结构，分别是虚拟地址空间 struct vmSPACE、虚拟地址区域 struct vmregion 和物理内存对象 struct pMobject。如图 4.14 所示，应用进程的虚拟地址空间由多个虚拟地址区域组成，每个虚拟地址区域关联一个物理内存对象，每个物理内存对象是

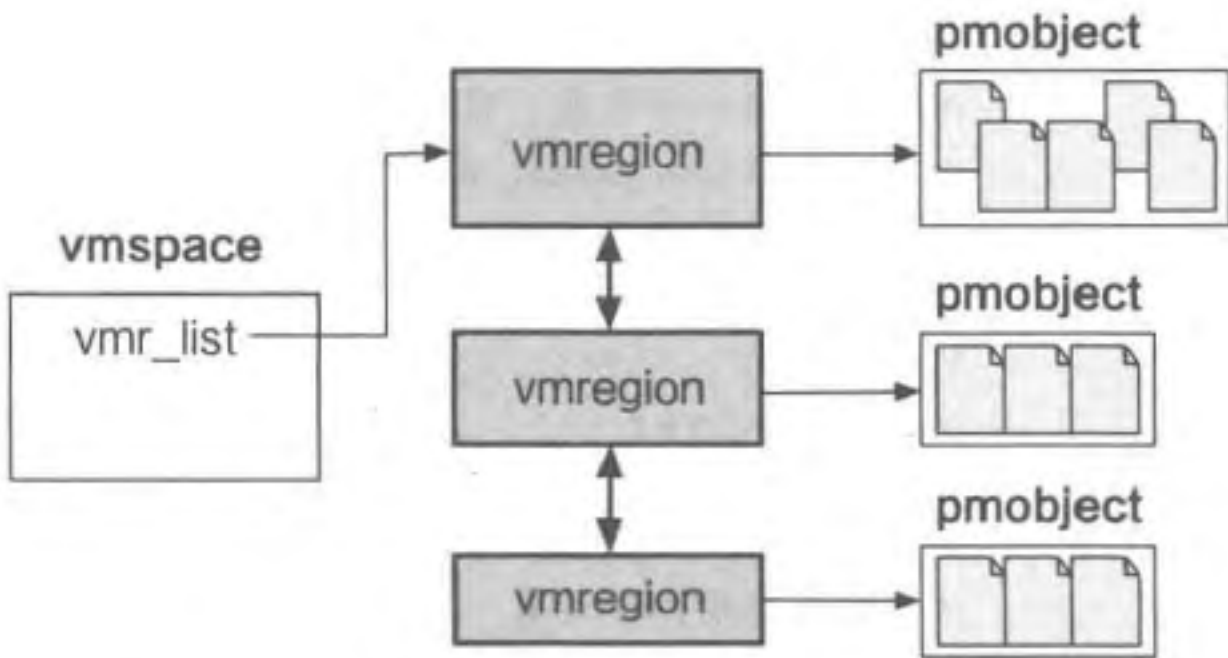


图 4.14 ChCore 内存管理中的三个重要数据结构

一些物理内存页的集合。

代码片段 4.13 给出了虚拟地址空间 `vmospace` 结构体中的三个成员变量。其中 `vmr_list` 是链表头，指向一个由虚拟地址区域组成的链表。应用进程的虚拟空间最大可达 2^{48} ，但是往往其中大部分都是未被使用的区域。ChCore 将一个应用地址空间内有效使用的虚拟地址区域保存在该链表里。`pgtbl` 存储着这个虚拟地址空间对应的页表基地址。需要注意的是，`pgtbl` 存储的地址是虚拟地址，由操作系统使用；对应的物理地址填在页表基地址寄存器中，由 MMU 使用。ChCore 在映射内核地址空间的时候，会保证内核虚拟地址和真正映射的物理地址之间相差一个固定的偏移量（直接映射）。因此，ChCore 很容易根据 `pgtbl` 这个虚拟地址计算出真正应该写入页表基地址寄存器中的物理地址（在 `vmospace` 中记录页表基地址的物理地址也是可行的，因为直接映射使虚拟地址和物理地址之间的转换变得很方便）。

代码片段 4.13 虚拟地址空间 `vmospace` 结构体

```

1 struct vmospace {
2     // vmregion 链表
3     struct list_head vmr_list;
4     // 页表基地址（虚拟地址）
5     vaddr_t *pgtbl;
6     // 用于修改地址空间的并发控制
7     struct lock vmospace_lock;
8
9     ...
10 };

```

代码片段 4.14 给出了虚拟地址区域 `vmregion` 结构体中的成员变量。其中，成员变量 `node` 将不同的 `vmregion` 对象连成链表，成员变量 `start` 和 `size` 分别标识该虚拟内存区域的起始地址和区域大小，成员变量 `perm` 标识该区域的读、写、执行权限，最后一个成员变量 `pmo` 标识映射到该区域的物理内存对象。实验内核采用简单的映射关系，只有一个物理内存对象映射到一个虚拟内存区域。

代码片段 4.14 虚拟地址区域 `vmregion` 结构体

```

1 struct vmregion {
2     // 链表中的一个节点，存储着 prev 和 next 指针
3     struct list_head node;
4     // 区域起始地址
5     vaddr_t start;
6     // 区域大小
7     size_t size;
8     // 访问权限
9     vmr_prop_t perm;
10    // 对应的物理内存对象
11    struct pobject *pmo;
12 };

```

ChCore 实验内核以对象的形式进行物理内存资源的管理。代码片段 4.15 给出了物理内存对象 `pobject` 结构体中的成员变量。`pobject` 对象分为不同的类型，由 `type` 成员标识，包含以下类型：

- `PMO_DATA`：课程实验中常用的类型，表示一段连续物理内存区域，即连续的物理页。在该类型中，成员变量 `start` 和 `size` 分别表示对应物理内存区域的起始地址和总大小。创建该类型的对象会立即进行物理内存分配，该类型的对象通常被用来映射给虚拟地址空间中的代码和数据区域，映射后不会发生缺页异常。
- `PMO_ANONYM`：该类型的物理内存对象同样表示物理页的集合，但是不要求物理页连续。分配该类型的对象不会立即触发物理内存分配，而是在访问时通过缺页异常进行按需分配，其中成员变量 `radix` 将记录所有分配的物理页。虚拟地址空间中的堆区域适合映射该类型的物理内存对象。
- `PMO_SHM`：该类型的物理内存对象用于实现应用进程之间的共享内存，即不同虚拟地址空间中的虚拟内存区域可以映射到同一个物理内存对象。成员变量 `refcnt` 表示对象的引用计数，当计数为 0 时，即可回收对应的物理内存资源。
- `PMO_USER_PAGER`：表示对应的虚拟内存区域由用户态进行管理。

其他的物理内存对象类型也各有用处，比如映射文件等，不过在目前版本的课程实验中并不会用到。

代码片段 4.15 物理内存对象 `pobject` 结构体

```
1 enum pmo_type{
2     PMO_DATA = 0,
3     PMO_ANONYM,
4     PMO_SHM,
5     PMO_FILE,
6     PMO_USER_PAGER,
7     ...
8 };
9 typedef enum pmo_type pmo_type_t;
10
11 struct pobject {
12     // 物理内存对象类型
13     pmo_type_t type;
14     // 根据类型使用 start 或 radix
15     union {
16         u64 start;
17         struct radix *radix;
18     } u;
19     // 对应的物理内存总大小
20     u64 size;
21     // 对象引用计数
22     atomic_cnt refcnt;
23 };
```


需要注意的是，PMO_USER_PAGER 是 pmo 对象的一个特殊类型。微内核的设计思想是将尽可能多的功能放到用户态，只在内核中保留必不可少的功能。因此，很多微内核将虚拟内存管理的功能放入用户态，内核只负责接收填写页表的系统调用，然后配置页表。用户态负责确定把哪些物理页映射给哪些虚拟页，甚至负责分配页表页（例如 seL4^[3]）。至于物理内存的分配，一般由运行在用户态的物理内存管理者负责（例如 Fiasco 中的 Sigma0^[4]）。但是，这样的设计可能会牺牲内存分配的性能，也可能导致缺页异常的开销变大等问题。于是，ChCore 采用了与 Google Zircon 微内核^[5]相似的设计，像宏内核一样，内核依然可以管理虚拟内存，不过同时保留了用户态管理虚拟内存的能力。

代码片段 4.16 列出了 ChCore 中两组关于虚拟内存管理的接口：分配虚拟内存区域和填写页表。vmSPACE_map_range 首先在虚拟地址空间 vmSPACE 中，根据参数起始虚拟地址 va 和区域长度 len 创建虚拟地址区域，然后将物理内存对象 pmo 与新创建的虚拟地址区域相关联。若物理内存对象 pmo 的类型是 PMO_DATA，vmSPACE_map_range 将继续调用 map_range_in_pgtbl 填写应用进程的页表。ChCore 使用伙伴系统进行（单个或连续多个）物理页的分配，也就是说，物理内存对象 pmo 中获得的物理页都是从伙伴系统中获得的。同时，ChCore 也实现了一个简化的 SLUB 分配器用于分配诸如 vmSPACE 等内核中的对象。第 5 章将介绍伙伴系统和 SLUB 分配器。

代码片段 4.16 内存管理相关接口

```

1 // 分配虚拟内存区域
2 int vmSPACE_map_range(struct vmSPACE *vmSPACE, vaddr_t va, size_t len,
   ↪ vmr_prot_t flags, struct pmo *pmo);
3 int vmSPACE_unmap_range(struct vmSPACE *vmSPACE, vaddr_t va, size_t len);
4
5 // 在页表中填写映射
6 int map_range_in_pgtbl(struct vmSPACE *vmSPACE, vaddr_t va, paddr_t pa,
   ↪ size_t len, vmr_prot_t flags);
7 int unmap_range_in_pgtbl(struct vmSPACE *vmSPACE, vaddr_t va, size_t len);

```

4.4 思考题

1. 单级页表需要占据连续的物理内存吗？为什么？
2. 在 AArch64 体系结构上，4 级页表结构中最后一级（第 3 级）页表项属性位的第 11 位是 nG（not Global）位。通过查阅硬件手册，我们可以知道当 nG 位是 0 时，表示该项映射对应的 TLB 缓存项对所有应用程序有效；反之，当 nG 位是 1 时，表示该项映射对应的 TLB 缓存项只对指定应用程序有效（根据 ASID）。请思考：为什么需要 nG 位？通常情况下 nG 位应设置成 0 还是 1？
3. 如果按照图 4.2 使用一张简单的单级页表来记录映射关系，那么对于 64 位的虚拟地址